

SUI Build Beyond

Delivers the benefits of Web3 with the ease of Web2



ON THE MOVE Thailand



ON THE MOVE Thailand



ON THE MOVE Thailand

Sui Overflow 2026

May 7 – Jun 20, 2026

\$500,000

Prize Pool

Headline Partner:

walrus

Track Sponsor:

 **DeepBook**

Register



DeepBook Predict & Sui Payment

Sui as a composable application layer

Sui is infrastructure for finance, payments, and programmable apps. DeepBook Predict and Sui Payment are two primitives on that layer — combine them and you unlock products that simply could not exist before.



Composable base layer

Shared infrastructure for finance, payments and on-chain logic.



DeepBook Predict

Part of the DeepBook financial stack — expiry-based markets.



Sui Payment

A payment stack — checkout, gas handling, receipts and on-chain settlement.



New opportunities

DeFi, payment apps, AI agents and on-chain financial tools.

01

SECTION

DeepBook Predict

Create markets around future outcomes — like whether an asset settles above a price, or inside a price range, at a set time. Oracle-priced, with shared vault liquidity.



What is DeepBook Predict?

A reusable market primitive for prediction, options-style and structured-market apps — so you build the experience, not the market engine.



Build markets, not market plumbing

Don't build pricing, vault or settlement — compose on shared on-chain state.

Builder note Testnet-stage today — package IDs & object layouts may change before Mainnet.



Oracle-driven markets



Binary positions



Vertical range positions



Structured products



Shared vault liquidity



Composable by design

Why DeepBook Predict matters



Less infra to build

You don't build core pricing, vault or settlement — spend your time on UX and product.



On-chain state & liquidity

Apps tap into shared on-chain state and pooled vault liquidity.



Composes with DeepBook

Combine with DeepBook Spot and Margin primitives.



New product surfaces

Express market views, risk exposure and structured positions.

POTENTIAL APPLICATIONS

Forecasting dashboards

Strategy builders

Portfolio risk tools

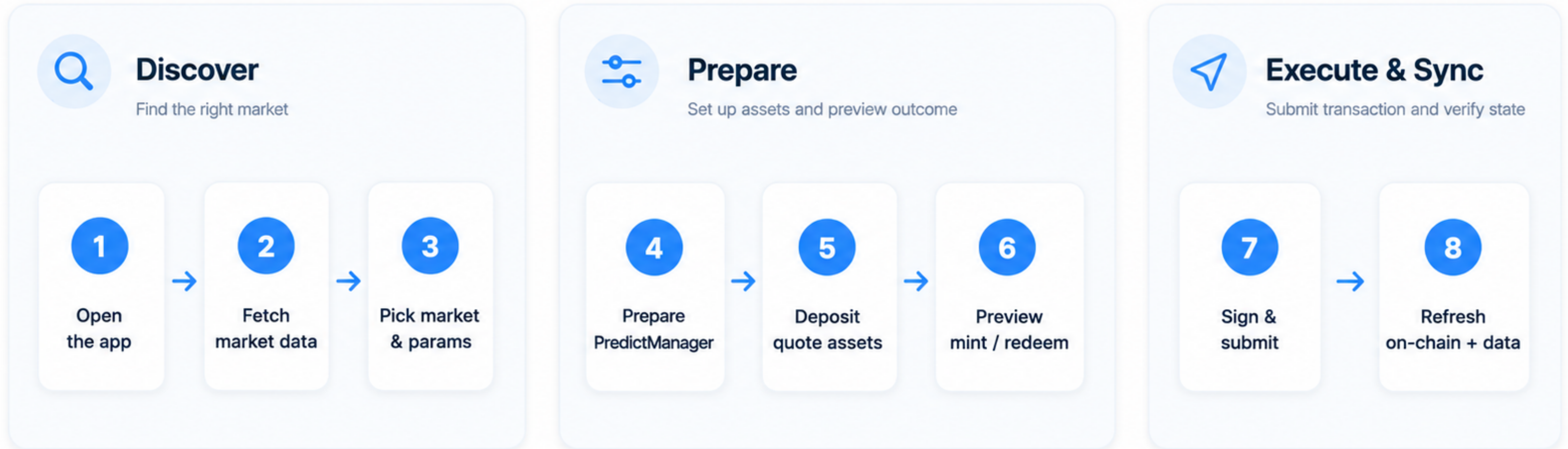
Structured products

LP vault analytics

DeepBook Predict vs a custom market

Topic	Custom prediction market	DeepBook Predict
Pricing	Built manually by each app	Oracle-based pricing + protocol risk controls
Liquidity	Fragmented per application	Shared vault liquidity
Position model	Custom logic per app	Binary & vertical range positions
Settlement	App-specific	Protocol-defined lifecycle
Builder experience	More complex	Easier to integrate & compose
Data layer	Must be built manually	Predict server, events & on-chain reads

A typical user flow



Two position types



Binary position

A simple yes / no on one price level.

BASED ON

- Oracle (which asset)
- Expiry (the time)
- Strike (the price level)
- Direction (above / below)

Example: Will SUI settle above \$5 at expiry?



Vertical range position

A view that the price lands inside a band.

BASED ON

- Lower strike
- Higher strike
- Expiry
- Oracle

Pays when settlement lands in the band: above lower, up to higher (lower, higher].

Example: Will SUI settle between \$4 and \$6 at expiry?

Oracle-priced --- not an order book or AMM

The market's price and status come from an oracle — the OracleSVI object — held per asset and expiry.

'A'

Where the price comes from

Fair value

+

Spread

+

Vault use

More likely → that side costs more.

NOT A CONSTANT-PRODUCT AMM



OracleSVI · the market's state

Its live price, status and final settlement price — per asset and expiry



Spot price



Forward price



Pricing curve (SVI)



Activation status



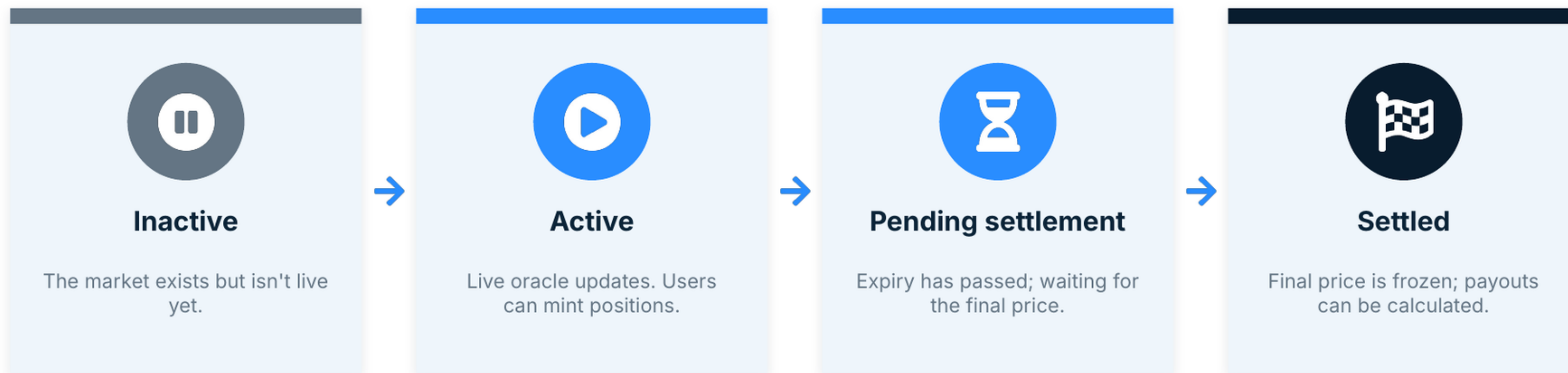
Timestamps



Settlement price

How a market moves through its life

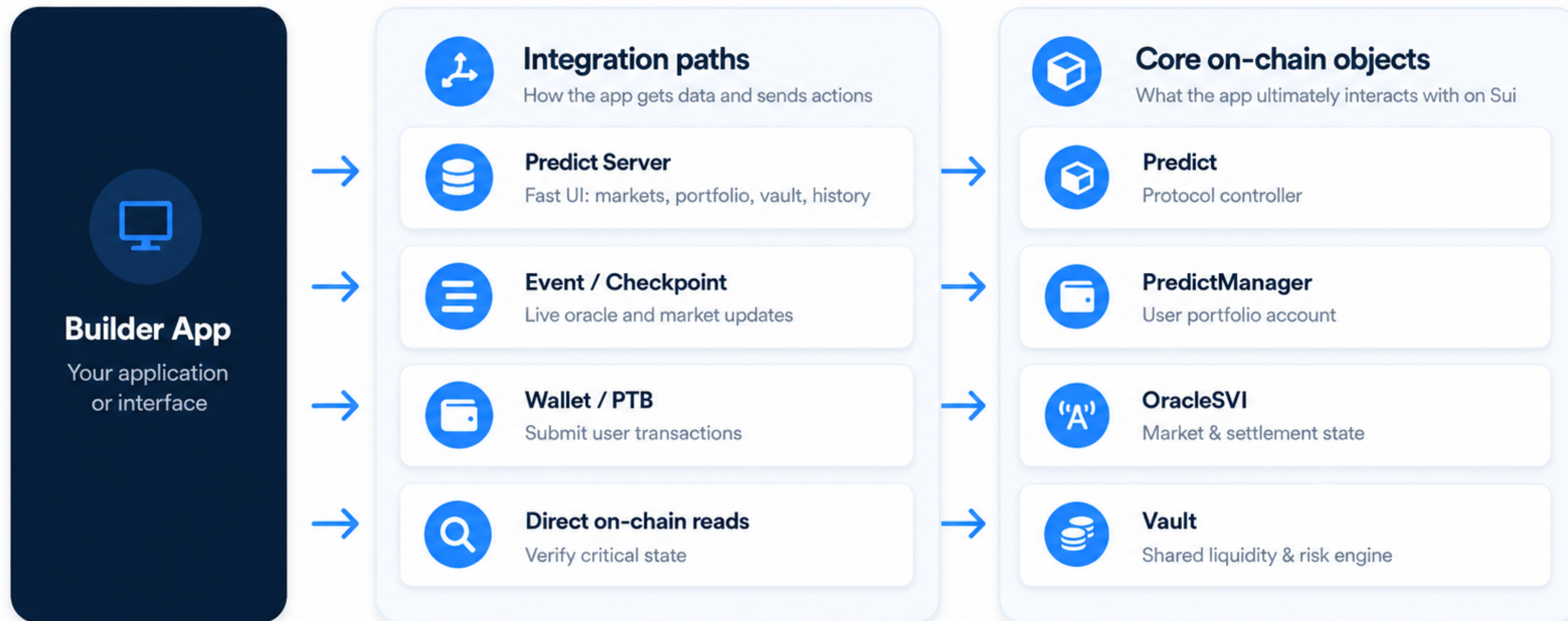
Every market passes through four stages — your UI should make the current stage obvious.



Key point: minting needs an Active oracle. After expiry, the first post-expiry price locks the settlement price.

How an app talks to Sui

A builder app uses four paths to interact with DeepBook Predict on Sui.



Main on-chain components

Component	What it is
Predict	Protocol controller — config, risk and trading rules.
PredictManager	Your portfolio account — balances and positions.
OracleSVI	Market & settlement state — price, status, settlement.
Vault	Shared liquidity & risk — exposure, payout, PLP shares.

Note: user positions are stored inside PredictManager — not as separate position objects.

The vault is the liquidity engine



The Vault takes the other side

Users mint or redeem against the Vault — no waiting for a matching user. If a position wins, the Vault pays out, so it manages exposure, liabilities and max payout.



Holds accepted quote assets



Counterparty to user positions



Tracks exposure & liabilities



Manages maximum payout risk



Lets LPs supply liquidity



Issues PLP — your LP share of the Vault

Data architecture for builders

Strong apps balance speed, reliability and trust — combine indexed data with direct on-chain verification.



Public Predict Server

Market list, vault summary, portfolio and historical data.

Fast UI



Event / Checkpoint Stream

Low-latency updates and live market events.

Live updates



Direct On-chain Reads

Critical state verification before & after wallet transactions.

Trust checks

What can builders build?

Start by visualizing the primitive — dashboards, simulators and explorers make great Testnet prototypes.



1

Market State Dashboard

Markets, oracle status, expiry timeline and vault health.



2

Range Strategy Simulator

Visualize range positions and payoff behavior.



3

Structured Product Simulator

Try payoff structures before touching the protocol.



4

Portfolio Risk Viewer

Open positions, exposure, PnL and expiry risk.



5

LP Vault Analytics

Vault value, liabilities, max payout and PLP performance.

Builder checklist

What your app should support before demo day



Market setup

- ✓ Market discovery & Predict Server
- ✓ Oracle lifecycle: Active → Settled



Portfolio & positions

- ✓ PredictManager & portfolio state
- ✓ Clear binary vs range UX



Execution flow

- ✓ Deposit, mint, redeem & settlement
- ✓ Wallet preview & confirmation



Verification & protection

- ✓ On-chain reads before & after tx
- ✓ Slippage / max-price protection



Liquidity & launch readiness

- ✓ Vault, PLP & max-payout display
- ✓ Testnet status & changing IDs

02

SECTION

Sui Payment

A programmable payment stack — address balances, payment intents, sponsored & gasless gas, and Payment Kit receipts. Web2-grade checkout with on-chain settlement.



More than transferring tokens

A token transfer just moves money. A payment system has to answer a lot more — and that is what Sui Payment is built for.



A token transfer

Answers one question:

Did funds move from A → B?



A payment system

Has to answer more:

- Which invoice or order was paid?
- Was the amount correct — and not paid twice?
- Who pays the gas?
- Where is the receipt, and how does it reconcile?

THE SUI PAYMENT STACK

Address Balances

Payment Intents

Sponsored & gasless gas

Payment Kit

Receipts & reconciliation

Why payment UX needs improvement

TODAY



Users need gas tokens



Multi-step flows



Receipts hard to track



Duplicate payments



Too technical for users



WITH SUI PAYMENT



Gasless / sponsored gas



One approval (Payment Intent)



Payment Kit: receipts + events



Built-in duplicate prevention



Web2-grade checkout

Core payment primitives

Not competing choices — these compose together into one checkout flow.



Address Balances

Hold funds as one balance per coin — like an account, not loose change. Simpler to send and pay.



Payment Intents

Bundle several steps into one transaction (PTB) the user approves once.



Sponsored Transactions

An app pays gas for the user; user and sponsor sign the same transaction.



Gasless Stablecoin Transfers

Eligible stablecoin transfers can go through without the user holding SUI for gas.



Payment Kit

Receipts, duplicate prevention, events and reconciliation — the merchant side.

Built-in payment logic

Attach conditions directly to the payment flow — move logic closer to the money, instead of relying only on backend middleware.



Release after verified delivery



Split between multiple recipients



Block on amount / recipient mismatch



Execute only when approved



Rules: expiry, limit, recipient, purpose



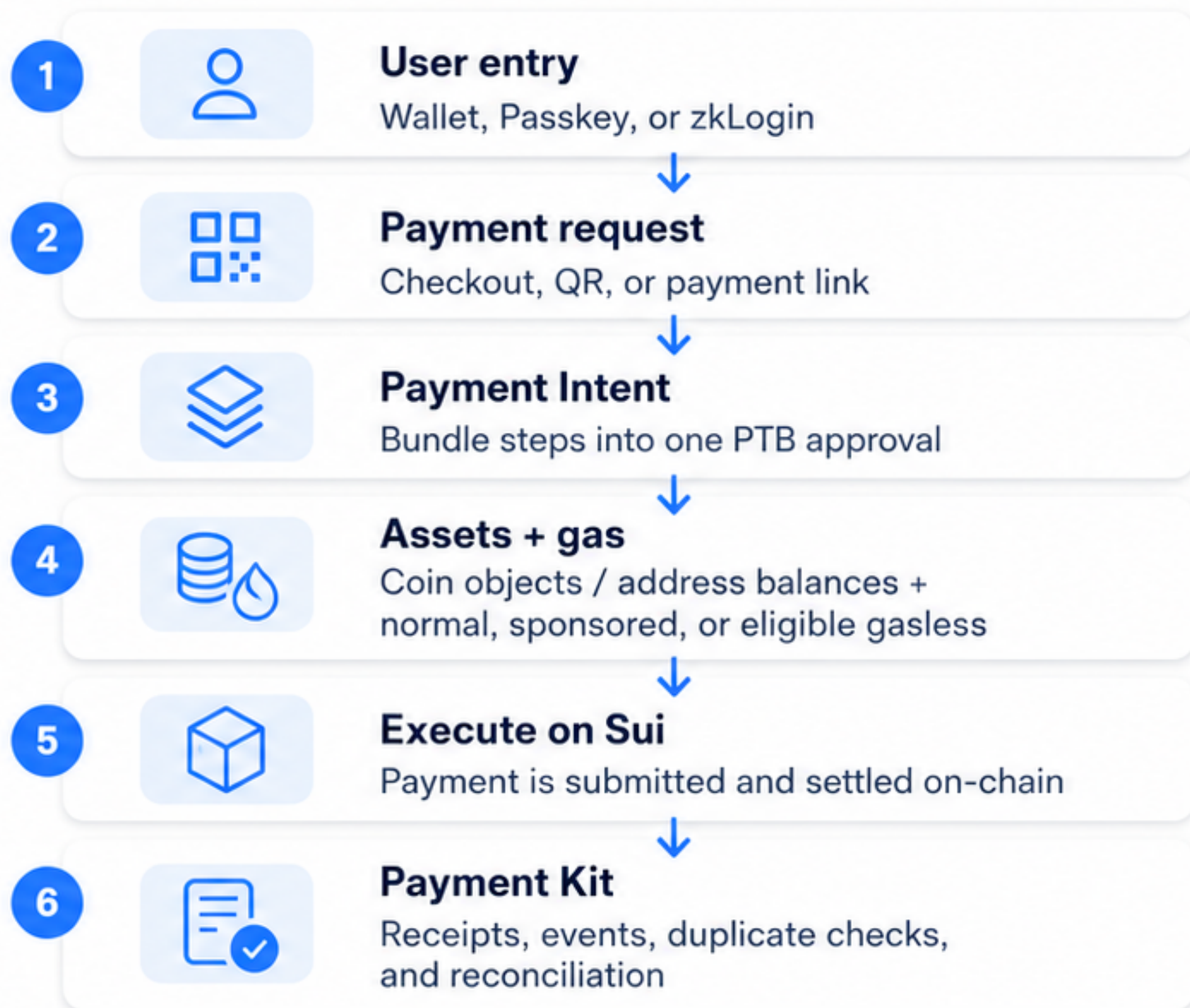
Compose conditions together

Note: real-world conditions (like delivery) need a trusted input — an oracle, backend attestation or approval signature.

Payment architecture

From user intent to on-chain settlement and merchant confirmation

HOW THE PAYMENT FLOWS



KEY BUILDER QUESTIONS

- 1** How does the user initiate payment?
- 2** Who pays the gas?
- 3** What asset is used to pay?
- 4** How is payment verified?
- 5** How does the merchant know it's done?
- 6** How are receipts and duplicates handled?



These questions shape your checkout UX and payment policy.

Sui Payment vs traditional rails

Topic	Traditional payment	Sui Payment
Settlement	Often delayed or batch-based	Near real-time, on-chain
Logic	Mostly backend-controlled	Programmable & composable
User experience	Mature but closed	Gasless or sponsored in supported flows; wallet UX
Reconciliation	External system required	Event & receipt-based via Payment Kit
Automation	Limited	Programmable; agent-ready with custom policy
Composability	Low	High

Sui provides the primitives — builders still design the payment policy, verification and reconciliation.

03

SECTION

Agentic Commerce

Delegated payment authority for AI agents — bounded, auditable, never unlimited. A builder pattern, enforced through app & wallet policy, Move capabilities and allowlists.



Delegated authority, not blank cheques



AI agents act on payments within predefined rules.

Agentic commerce should never mean unlimited control of funds. It means delegated payment authority with clear, enforceable constraints.

CONSTRAINTS YOU CAN ENFORCE



Spending cap



Allowed recipient



Payment purpose



Expiry time



Approval threshold



Asset type

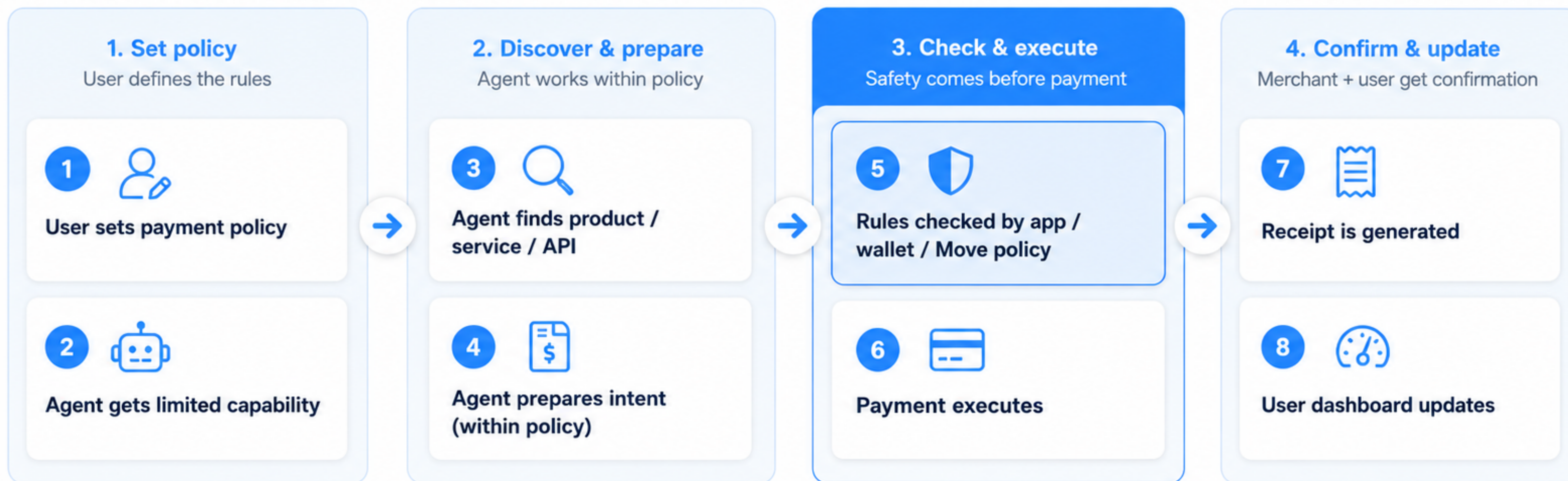


Frequency limit

Enforced by app & wallet policy, Move capabilities, allowlists and approval signatures — not automatic.

How an agentic payment runs

A safe flow for delegated payments with clear limits, checks, and receipts.



For high-value actions, the agent can prepare the payment — but the user should still sign the final step.

Where agents pay within limits



SaaS Credit Top-up

Tops up credits within a set budget.

Uses: Payment Kit + cap + receipts



Autonomous API Payment

Pays for data or inference per use.

Uses: Ephemeral payments + events



Travel / Event Assistant

Books or pays within approved limits.

Uses: Policy + user approval



Creator Micropayments

Pays creators or data providers per use.

Uses: Gasless if eligible, else sponsored



DAO Treasury Assistant

Prepares payment batches for approval.

Uses: Batch PTB + multisig approval

04

SECTION

Build Ideas

Real project directions, curated from the Sui ecosystem (sui.io/project-ideas) — DeepBook & DeFi markets, payment-driven apps, and a combined AI build.



Ideas that fit DeepBook Predict

From the Sui ecosystem's project ideas — markets you can build on oracle-priced, vault-backed liquidity.



Prediction Market

Markets on events, sports or price trends, settled on-chain.

Revenue: Platform & event fees



On-Chain Options

Buy and sell call / put options, priced via DeepBook.

Revenue: Execution fees • analytics



DeFi Derivatives Exchange

Perps, futures and synthetics on DeepBook liquidity.

Revenue: Trading • funding fees



BTC Yield Markets

Trade BTC yield, Pendle / Spectra style.

Revenue: Trading • yield products



AI Trading Bots

Automated market-making & portfolio bots on DeepBook.

Revenue: Subscription • perf fee

Curated from sui.io/project-ideas

Apps that ride on Sui Payment

From the Sui ecosystem's project ideas — consumer apps that live or die on smooth payment UX.



Robinhood for Crypto

A friendly trading app that hides DeFi complexity.

Revenue: Transaction fees • subs



Simplified DeFi Super App

Yield, staking and lending as easy as Web2 banking.

Revenue: Fees • staking share



Creator Tipping App

Twitch / Patreon-style — tip creators in crypto.

Revenue: Tip fees • subs • ads



GoFundMe for Web3

Crowdfunding for Web3 startups and real-world causes.

Revenue: Donation fees



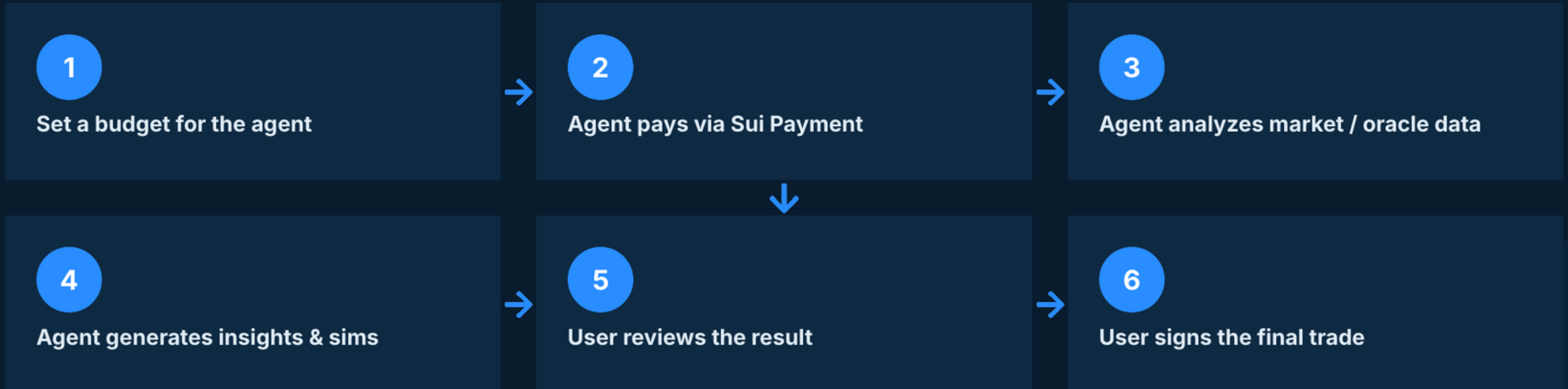
Crypto Tax Software

TurboTax-style tax & accounting for Sui activity.

Revenue: Subscription • pro tier

Curated from sui.io/project-ideas

AI Research Agent + Sui Payment + DeepBook Predict



Payment pieces used: *budget policy · Payment Intent · Payment Kit receipt · reconciliation — and the user signs the final financial action.*

Key takeaways



1

DeepBook Predict = a market primitive

Oracle-priced markets · binary & range positions · shared vault.



2

Sui Payment = a full payment stack

Address balances · intents · gasless gas · Payment Kit receipts.



3

Design for user experience

Not just smart-contract integration.



4

Blend your data sources

Indexed data · event streams · on-chain verification.



5

Overflow 2026: ship a prototype

Clear use case · simple UX · working demo.

Q&A

Questions, ideas, or want to build together for SUI Overflow 2026?

[Explore the docs](#) · [Join the demo session](#) · [Reach out to the team](#)